

# RapidTriage: A Hybrid LLM and Rule-Based Architecture for Safety-Biased Emergency Symptom Triage

Kaustubha Venkata Eluri  
Northeastern University  
eluri.k@northeastern.edu

**Abstract**—Large language models (LLMs) are increasingly proposed as front-line interfaces for high-stakes decision support, yet their well-documented tendency toward hallucination and inconsistent behavior makes them unsuitable as the sole arbiter of decisions with patient-safety consequences. This paper documents RapidTriage, a cross-platform (React Native/Expo and Go) medical symptom triage application built for a graduate artificial intelligence course, which addresses this problem through a hybrid architecture: an LLM is used only to parse unstructured natural-language symptom descriptions into a typed, structured representation, while a deterministic, auditable rule engine – structurally inspired by, but not certified against, the Manchester Triage System (MTS) and Emergency Severity Index (ESI) – performs the actual triage classification. We describe the typed-contract boundary that isolates LLM non-determinism from the classification logic, a confidence-threshold fallback mechanism that explicitly biases uncertain classifications toward greater urgency (an asymmetric Type-I/Type-II error trade-off), and backend engineering choices in Go including bounded-concurrency tool dispatch, deadline-bounded graceful shutdown, and multi-provider LLM dispatch across Gemini, Claude, GPT-4o, and Llama. We report the project’s own classroom-scale evaluation (85–92% per-category accuracy, ~3.7s average response time) and, following the project’s own documentation, explicitly frame this evaluation, the mocked external tool integrations, and the non-certified rule set as first-class, disclosed limitations rather than concealed shortcomings. We argue that this disclosure practice – treating validation gaps as documented engineering artifacts – is itself a contribution to how hybrid AI safety-critical systems should be reported.

**Index Terms**—hybrid AI systems, neuro-symbolic architectures, clinical decision support, large language models, triage, safety-critical software, human-AI reliability

## I. INTRODUCTION

Emergency symptom triage is a high-stakes classification problem: a decision that under-estimates urgency risks patient harm, while a decision that over-estimates urgency wastes scarce clinical resources. Manual triage protocols such as the Manchester Triage System (MTS) and the Emergency Severity Index (ESI) formalize this problem as a deterministic decision tree over structured clinical features (symptom severity, anatomical location, duration, and accompanying symptoms) so that the same inputs reliably produce the same triage code.

Large language models are attractive as a front-end to such systems because they can parse free-text, conversational symptom descriptions – something rigid decision trees cannot do directly – and can ask clarifying questions to disambiguate

vague complaints. However, LLMs are known to hallucinate, to be sensitive to prompt phrasing, and to lack any built-in mechanism for guaranteeing a bounded, auditable output space. Using an LLM as the sole decision-maker for a task with irreversible real-world consequences is therefore difficult to justify: there is no way to audit an LLM’s classification logic post hoc, and its confidence (when it produces one) does not carry the same guarantees as a designed decision procedure.

RapidTriage, a project built for CS 5100 (Artificial Intelligence) at Northeastern University, addresses this tension with a two-stage hybrid design: an LLM converts unstructured user input into a structured intermediate representation, and a separate, deterministic rule-based classifier consumes that structure to produce a triage code. This paper documents the architecture, the safety-biased design of the classifier’s fallback behavior, the backend engineering used to make the system responsive and resilient, and the project’s own documented evaluation and limitations. Consistent with the project’s own architecture decision records (ADRs) and limitations documentation, we do not claim clinical validation, FDA/MDR certification, or production-grade reliability; rather, we document these gaps as they are recorded in the project, and argue that transparent limitation-reporting is itself part of responsible engineering practice for AI-assisted clinical-adjacent tools.

The remainder of this paper is organized as follows. Section II situates RapidTriage relative to general work on hybrid neuro-symbolic systems and clinical decision support. Section III describes the system architecture and the typed contract between the LLM and rule engine. Section IV details the safety-biased confidence-threshold fallback and its Type-I/Type-II error rationale. Section V covers backend engineering decisions in Go. Section VI reports the project’s documented evaluation results. Section VII discusses documented limitations, and Section VIII concludes with future work drawn from the project’s roadmap.

## II. RELATED WORK

### A. Hybrid Neuro-Symbolic Systems

A recurring theme in AI systems that must satisfy both flexibility and correctness requirements is the separation of a statistical, pattern-matching component from a symbolic, rule-governed component. Neuro-symbolic architectures generally

place a learned model at the perception or parsing boundary and a symbolic reasoner at the decision boundary, on the premise that the learned component’s uncertainty should not propagate uncontrolled into decisions that require auditability or formal guarantees. RapidTriage instantiates this pattern narrowly: the LLM’s role is restricted to information extraction (unstructured text  $\rightarrow$  structured fields), and all classification logic downstream is symbolic and deterministic.

### B. Clinical Decision Support Systems

Clinical decision support systems (CDSS) have historically relied on rule-based or scoring-based approaches (e.g., decision trees, weighted scoring rules) precisely because clinical and regulatory environments demand explainable, reproducible, and auditable decision paths. The introduction of LLMs into this space raises the question of how to gain the language-understanding benefits of LLMs without forfeiting the auditability that rule-based CDSS provide. RapidTriage’s design – confining the LLM to a parsing role and keeping classification rule-based – reflects this general design tension observed across the CDSS literature, without claiming any specific certified clinical framework: the project explicitly describes its rule set as *MTS/ESI-inspired*, not *MTS/ESI-certified*.

### C. Positioning

Unlike systems that treat an LLM’s own confidence or chain-of-thought as sufficient justification for a decision, RapidTriage treats the LLM strictly as an unreliable sensor whose output must be validated and reduced to a bounded schema before any consequential decision is made from it. This positions the project’s contribution less as a modeling advance and more as a systems and engineering pattern for safely composing LLMs with deterministic decision logic in a domain where errors are asymmetric in cost.

## III. SYSTEM ARCHITECTURE

### A. Request Flow

RapidTriage is a client-server system: a React Native (Expo) mobile client captures symptom descriptions as text or audio and communicates over HTTPS with a Go backend. The end-to-end flow, as documented in the project’s architecture notes, proceeds as follows:

- 1) The client submits a request to `POST /api/v1/emergency/text` (text) or `POST /api/v1/emergency` (multipart audio, capped at a configurable `MAX_AUDIO_SIZE_MB`, default 20MB).
- 2) An `EmergencyHandler` dispatches the request to an `AudioProcessor` or `TextProcessor`, which invokes the configured LLM provider (`ModelType`, defaulting to Gemini).
- 3) The LLM’s output is returned not as free text but as a structured `EmergencySituation` value.
- 4) A coordinator submits this structure to `RuleBasedClassifier.Classify(ctx,`

`situation)`, which returns a triage code, a confidence score, and an error value.

- 5) If the returned confidence is below a configured threshold, a safety fallback code is substituted (Section IV).
- 6) Downstream tools (hospital lookup, ambulance dispatch, location capture, notification) are invoked concurrently, and a structured response is returned to the client containing the triage code, confidence, a human-readable summary, recommended tools, hospital alerts, an emergency identifier, and a timestamp.

### B. The Typed Contract Between LLM and Rule Engine

The central architectural decision in RapidTriage is the presence of a strict, statically typed boundary between the LLM’s output and the rule engine’s input. This boundary is realized as the `EmergencySituation` struct: a fixed schema of clinically relevant fields (e.g., severity score, anatomical location, duration, accompanying symptoms) that the LLM is prompted to populate, and which is the *only* channel through which LLM output can influence the triage decision. The rule engine itself is defined behind a narrow interface:

```
type Classifier interface {
    Classify(ctx context.Context,
            situation *models.EmergencySituation,
            ) (models.TriageCode, float64, error)
}
```

This design has two consequences that the project’s own documentation makes explicit. First, it bounds the *shape* of what the LLM can influence: regardless of how the LLM phrases its reasoning, it cannot cause the system to skip the classifier or invoke an unbounded action space – it can only fill in fields of a fixed schema. Second, and importantly, the project’s limitations documentation is candid that this schema constrains output *shape*, not output *factual content*: a well-formed but factually wrong `EmergencySituation` (e.g., a hallucinated severity score) is not prevented by the type system, only downstream mitigated by the confidence-threshold fallback described next. This distinction – structural safety versus semantic correctness – is a useful general lesson for any system that interposes a typed contract between an LLM and a decision procedure: typing constrains the parser, not the parser’s fidelity to the truth.

Four triage codes are produced by the classifier: `IMMEDIATE`, `VERY URGENT`, `URGENT`, and `NON-URGENT`, mirroring the ordinal structure of MTS/ESI-style protocols without adopting their licensed decision tables.

The overall data and control flow described above is summarized in Fig. 1, which depicts the client-server boundary, the LLM parsing stage, the typed contract, and the rule-based classification stage as separate architectural layers.

## IV. SAFETY-BIASED DESIGN

A defining engineering decision in RapidTriage, recorded in the project’s architecture decision records, is that classifier uncertainty is resolved asymmetrically: when the rule-based

1. Input → 2. LLM clarification → 3. Rule engine → 4. Output → 5. Alerts

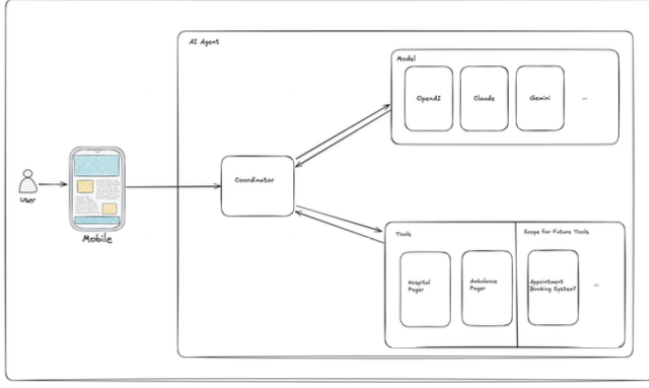


Fig. 1. RapidTriage system architecture, showing the React Native/Expo client, the Go backend’s request handling and LLM provider dispatch, the typed `EmergencySituation` boundary, the rule-based classifier, and the concurrently invoked downstream tools.

classifier’s confidence falls below a configured threshold (default 0.5, with confidence normalized to  $[0, 1]$ ), the system does not default to the lowest-urgency code. Instead, it falls back to a configurable `FallbackCode`, set by default to `YELLOW` (Urgent) rather than to a low-urgency category, and rather than to the highest-urgency category.

#### A. Type-I/Type-II Error Framing

The project’s own decision record frames this choice directly in terms of the classic asymmetric-cost hypothesis-testing trade-off:

- A **Type-I error** in this context – escalating a case that was not actually urgent – wastes emergency-room triage capacity.
- A **Type-II error** in this context – classifying a genuinely urgent case as non-urgent – risks direct patient harm.

Because the cost of a Type-II error dominates the cost of a Type-I error in a triage setting, the system is deliberately designed to prefer over-triage to under-triage whenever the classifier is not confident in its result. This is a textbook application of asymmetric decision-cost reasoning to system design: rather than treating classifier uncertainty as noise to be resolved arbitrarily (e.g., by defaulting to the majority class or the lowest-urgency class), RapidTriage encodes the clinical intuition that erring toward urgency is the safer failure mode.

#### B. Bounded Escalation

Notably, the fallback does not escalate all the way to the highest-urgency code (`RED/IMMEDIATE`) on low confidence. The project’s rationale, again drawn directly from its decision records, is that escalating every low-confidence case to the maximum urgency level would trigger heavy, resource-intensive automation (e.g., unnecessary ambulance dispatch) disproportionate to genuine uncertainty. Instead, the intermediate `YELLOW/Urgent` fallback recommends that the user seek care soon, without triggering the most resource-intensive downstream actions. This reflects a more nuanced

safety design than a naive “always escalate to maximum” heuristic: the fallback level is chosen to bound the *cost* of the safety bias, not merely to maximize caution unconditionally.

### V. BACKEND ENGINEERING

The Go backend was engineered with several concrete reliability mechanisms, documented in the project’s architecture decision records.

#### A. Concurrent Tool Dispatch

In a high-urgency case, several independent downstream actions may need to occur together: paging the nearest hospital, dispatching an ambulance, capturing the user’s geocoded location, generating a human-readable summary, and sending an SMS notification. The project’s decision record estimates that each such tool call takes on the order of 500ms to 2s, so that fully serial execution could add up to roughly 7 seconds of latency versus roughly 2 seconds if the calls are parallelized. RapidTriage dispatches these tool calls concurrently, bounded by a configurable `MaxConcurrentTools` parameter (default 5), implemented with a goroutine-and-`WaitGroup` pattern rather than a callback chain, with a mutex protecting writes to the shared result map.

#### B. Graceful Shutdown

The server wires `signal.NotifyContext` against `SIGINT` and `SIGTERM`, and on receipt of either signal calls `server.Shutdown(ctx)` with a 10-second deadline before the process exits. The rationale documented in the project’s decision records is explicit: an in-flight emergency request must not be abruptly terminated by a deployment or process-management signal, and 10 seconds is judged sufficient to drain a typical in-flight request while still bounding how long a stalled request can delay shutdown.

#### C. Timeouts

The HTTP server bounds all requests with a `ReadTimeout` of 30s, a `WriteTimeout` of 30s, and an `IdleTimeout` of 120s. Calls to AI providers are separately bounded by a configurable per-call timeout (documented as `API_TIMEOUT_SECONDS` in the README and as `AI_MODEL_TIMEOUT_SECONDS` in the architecture notes, both defaulting to 30s), and audio uploads are capped at a configurable size limit (default 20MB). These bounds ensure that a slow or unresponsive third-party LLM provider cannot indefinitely stall a client-facing emergency request.

#### D. Multi-Provider Dispatch

The backend abstracts over four LLM providers – Gemini (default, `gemini-1.5-pro`), Claude (`claude-3-opus-20240229`), OpenAI (`gpt-4o`), and Llama (self-hosted, per-host configuration) – behind a single `Provider` interface, selected at startup via an `AI_MODEL_TYPE` environment variable, with provider-specific endpoint/key/model environment variables and a set of general-purpose overrides that take priority when set. This configuration-driven dispatch is motivated, per the

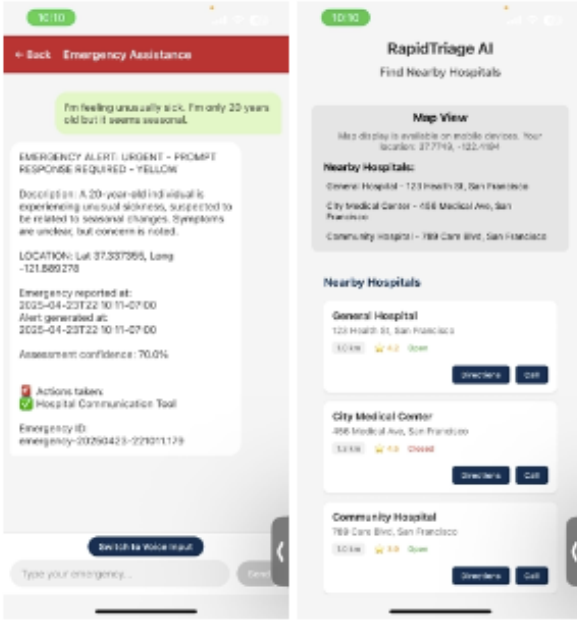


Fig. 2. RapidTriage client screens used during the classroom-scale evaluation, showing the symptom-entry interface and the resulting triage summary presented to the user.

project’s decision records, by the observation that a single provider’s outage or rate-limiting should not be capable of blocking emergency triage entirely, though we note – consistent with the project’s own documentation – that the system as documented implements provider *selection*, not automatic runtime failover between providers with retry/backoff semantics; that mechanism is left to future work (Section VIII).

## VI. EVALUATION

RapidTriage’s own documented evaluation is a classroom-scale test rather than a statistically powered clinical study, and we report it here exactly as documented, with the same caveat the project itself attaches. The reported results are:

- Average end-to-end response time: approximately 3.7 seconds, against an internal target of under 5 seconds.
- Overall triage accuracy: 85–92%.
- Per-category accuracy: chest pain 92%, leg pain 90%, stomach ache 88%, headache 85%, light discomfort 80%.
- False positive rate: described in the project documentation as “monitored, threshold-calibrated” rather than reported as a single quantitative rate.

The project’s documentation attributes the higher accuracy on presentations such as chest pain and leg pain to clearer, more well-defined symptom patterns, and the comparatively lower accuracy on categories such as headache and light discomfort to greater reliance on iterative clarifying questions to disambiguate vague complaints.

Fig. 2 shows the React Native/Expo client screens exercised during this evaluation, illustrating the symptom-input flow and the triage result summary returned to the user.

Critically, the project’s own limitations documentation states plainly that these numbers “come from a few-dozen-sample classroom test run, not a statistically rigorous evaluation,” and that they are “illustrative of order-of-magnitude behavior, not production guarantees.” We adopt this framing without modification: no macro-F1 score, precision/recall breakdown against a clinically validated ground truth, or inter-rater agreement against trained triage nurses is reported anywhere in the project’s documentation, and we do not construct or estimate one. Any such metric would misrepresent the maturity of the evaluation actually performed.

## VII. LIMITATIONS

Consistent with the strict-disclosure principle underlying this paper, we report RapidTriage’s limitations exactly as documented in the project rather than paraphrasing them into stronger claims.

### A. Non-Certified Clinical Framework

The rule-based classifier is explicitly documented as MTS/ESI-*inspired*, not MTS/ESI-*certified*: the canonical MTS and ESI rule sets are licensed and require formal clinical training to administer, and the project adopted only their general decision-tree structure as a pedagogical reference rather than licensing or implementing the canonical rule tables. The project documentation states directly that the system carries no FDA, MDR, or other medical-device certification, and has not undergone a HIPAA or GDPR compliance audit, and is explicitly “not suitable for clinical deployment” in its current form.

### B. Mocked Tool Integrations

All four downstream tools – location lookup, hospital paging, ambulance dispatch, and appointment booking – are implemented against a mock HTTP client that returns hardcoded JSON responses rather than integrating with real hospital paging systems, dispatch services, or scheduling platforms (e.g., Cerner/Epic/Athena). The project documentation is explicit that “the agent code is correct; the integrations are not real,” framing this as a scoping decision appropriate to a one-semester, two-person academic project rather than a defect in the agent orchestration logic itself.

### C. LLM-Specific Limitations

The project documents standard LLM risk factors that remain unmitigated beyond the structural typed-contract boundary: hallucination risk in populating the EmergencySituation schema, no clinical fine-tuning of any of the four supported models, no multilingual support at the time of writing, and no on-device inference (all inference is via hosted provider APIs).

#### D. Operational and Privacy Gaps

The documented backend has no structured logging (relying on `log.Println`), no telemetry or metrics export, no rate limiting on the emergency-intake endpoint, and no load testing. On the mobile client, the project documents no offline mode, a single-user/stateless design, no client-side encryption or anonymization of symptom descriptions prior to the LLM call beyond transport-level HTTPS, and no accessibility audit.

We regard the explicit, itemized documentation of these gaps – rather than their omission – as evidence of sound engineering practice for a project of this scope, and structure this paper’s Limitations section to preserve that transparency rather than to soften it.

### VIII. CONCLUSION AND FUTURE WORK

RapidTriage demonstrates a practical pattern for incorporating LLMs into a safety-relevant decision pipeline without allowing LLM non-determinism to directly govern the decision: an LLM is confined to a parsing role behind a fixed typed schema, and the actual triage decision is made by a deterministic, auditable rule engine with an explicitly safety-biased fallback for low-confidence classifications. The project’s Go backend additionally demonstrates concrete reliability engineering practices – bounded-concurrency tool dispatch, deadline-bounded graceful shutdown, and multi-provider configuration – appropriate to a latency- and availability-sensitive request path. At the same time, the project’s own documentation is candid that its evaluation is classroom-scale rather than clinically validated, that its rule set is inspired by but not certified against MTS/ESI, and that its external tool integrations are currently mocked.

The project’s own roadmap lays out a path toward closing these gaps, and we summarize it here without extension: replacing the mocked tool integrations with real hospital-, dispatch-, and scheduling-system partnerships; conducting a clinically supervised validation study (with IRB approval) comparing classifier output against trained-nurse triage decisions at scale, together with adversarial testing of the escalation behavior described in Section IV; distilling the hosted LLMs into a smaller on-device model to enable offline operation; adding multilingual and accessibility support; introducing persistent, authenticated user profiles; hardening the operational surface with structured logging, metrics, and rate limiting; and pursuing an appropriate regulatory pathway (e.g., FDA Class II Software as a Medical Device analysis and a HIPAA Security Rule audit) before any real-world deployment. The project’s documentation is explicit that replacing professional nurse triage, providing telemedicine consultations, and handling insurance claims are intentionally out of scope. We view this roadmap, together with the disclosure practices documented in Sections VI and VII, as the appropriate way to report an academic prototype whose architectural pattern – typed LLM/rule-engine separation with asymmetric-cost fallback – may be of independent interest even where the specific clinical instantiation is not yet validated.

#### ACKNOWLEDGMENT

The author thanks collaborator Yadhukrishnan Pankajashan, who contributed backend logic, rule engine, API development, and the alert system for the RapidTriage project, developed for CS 5100, Artificial Intelligence, at Northeastern University.